

TRƯỜNG ĐẠI HỌC BÁCH KHOA – ĐHQG-HCM

Bộ môn Khoa học Máy tính

Học phần: Introduction to Artificial Intelligence

Giảng viên hướng dẫn: TS. Trương Vĩnh Lân

BÁO CÁO BÀI TẬP LỚN

Hệ thống Tối ưu hóa Định tuyến Giao hàng AI

Thành viên nhóm	MSSV
Vạn Trường Thành	2353107
Lê Minh Trí	2353229
Nguyễn Minh Khôi	2352618
Trần Nguyễn Nhất Thông	2353149

Mục lục

1	Mô tả Bài toán Thực tế	3
1.1	Bối cảnh và Động lực	3
1.2	Phát biểu Bài toán	3
1.3	Các Yếu tố Bất định	3
2	Mô hình AI	3
2.1	Biểu diễn Trạng thái	4
2.2	Knowledge Representation – IF-THEN Rules	4
2.3	Uncertainty Modeling – Mạng Bayes	5
2.4	Mô hình Học máy	6
3	Các Thuật toán Sử dụng	7
3.1	Thuật toán Tìm đường – A^*	7
3.2	Giải quyết TSP – Brute-force Permutation + CSP	7
3.3	Mạng Bayes – Conditional Probability Tables	8
3.3.1	CPT cho RoadCondition	8
3.3.2	CPT cho TrafficDemand	8
3.3.3	EffectiveCapacity và AccidentRisk	9
3.3.4	CPT cho Congestion	9
3.3.5	Công thức suy luận	9
4	Thực nghiệm	10
4.1	Thiết lập Thực nghiệm	10
4.2	Phân tích Dữ liệu (EDA)	10
4.3	Kết quả Thực nghiệm	11
4.3.1	Kết quả Logic Filter	11
4.3.2	Kết quả Mạng Bayes Realtime	11
4.3.3	Kết quả So sánh Mô hình ML	12
4.3.4	Kết quả A^* + CSP (Time Windows)	13
4.3.5	Kết quả Tích hợp & Re-planning	13
5	Phân tích Kết quả	14
5.1	Phân tích Feature Importance	14
5.2	Vai trò của Bayes Logic trong Pipeline	14
5.3	Phân tích Loss Curve	15
5.4	Hiệu quả Re-planning	16

6	Kết luận	16
6.1	Tóm tắt Đóng góp	16
6.2	Hạn chế và Hướng phát triển	16
7	Phân công Công việc	17
8	Mã nguồn & Tài nguyên	17

1. Mô tả Bài toán Thực tế

1.1. Bối cảnh và Động lực

Trong lĩnh vực logistics và giao hàng chặng cuối (*last-mile delivery*), một trong những thách thức lớn nhất là lập lộ trình tối ưu cho xe giao hàng khi phải đến nhiều điểm đích trong khoảng thời gian nhất định, đồng thời đối phó với các yếu tố bất định như kẹt xe và thời tiết xấu. Đây là phiên bản thực tế của bài toán **Vehicle Routing Problem with Time Windows (VRPTW)**.

Nhóm đã chọn khu vực **DH Bách Khoa – Lý Thường Kiệt, Quận 10, TP.HCM** làm môi trường thử nghiệm. Dữ liệu bản đồ được tải trực tiếp từ OpenStreetMap thông qua thư viện OSMnx, đảm bảo tính thực tế của môi trường thử nghiệm.

1.2. Phát biểu Bài toán

Bài toán được phát biểu như sau:

- Cho một xe giao hàng (tải trọng 2.5 tấn) xuất phát từ điểm depot (DH Bách Khoa) lúc 8:00 sáng (480 phút).
- Xe cần giao hàng đến $N = 4$ điểm đích, mỗi điểm có khung giờ yêu cầu [open, close].
- Tình trạng giao thông (kẹt xe) và thời tiết thay đổi trong quá trình giao hàng.
- **Mục tiêu:** Tìm thứ tự giao hàng tối ưu để tổng thời gian di chuyển nhỏ nhất, thỏa mãn tất cả ràng buộc khung giờ.
- Hệ thống phải có khả năng tính lại lộ trình (*re-planning*) khi điều kiện môi trường thay đổi.

1.3. Các Yếu tố Bất định

Hệ thống xử lý hai yếu tố bất định chính:

- **Thời tiết (Weather):** `clear` / `rain` – ảnh hưởng đến tốc độ di chuyển và xác suất kẹt xe.
- **Khung giờ (Time of Day):** `normal` / `peak` – giờ cao điểm làm tăng đáng kể xác suất kẹt xe, đặc biệt trên các trục đường chính.

2. Mô hình AI

2.1. Biểu diễn Trạng thái

Bảng 1: Biểu diễn bài toán dưới dạng bài toán tìm kiếm

Thành phần	Mô tả
State	(current_location, current_time, remaining_deliveries)
Action	Di chuyển từ node hiện tại đến node kế tiếp trên đồ thị đường bộ
Goal	remaining_deliveries = $\emptyset$ (đã giao xong tất cả điểm)
Cost	travel_time_min – thời gian di chuyển thực tế dự đoán bởi mô hình ML

2.2. Knowledge Representation – IF-THEN Rules

Trong hệ thống, tri thức giao thông được biểu diễn dưới dạng các luật suy diễn IF-THEN và được hiện thực trong module `logic_filter.py`. Mục tiêu của module này là lọc đồ thị đường bộ trước khi thuật toán tìm kiếm A* được thực thi, đảm bảo xe giao hàng chỉ di chuyển trên các đoạn đường hợp lệ về mặt dữ liệu, quyền truy cập, tải trọng và trạng thái hoạt động của đường.

Dữ liệu bản đồ sau khi được tải từ OSMnx được biểu diễn dưới dạng `nx.MultiDiGraph`, trong đó mỗi cạnh biểu diễn một đoạn đường có hướng. Mỗi cạnh có thể chứa các thuộc tính như `length`, `access`, `vehicle`, `motor_vehicle`, `hgv`, `goods`, `maxweight`, `maxgcweight`, `highway`, `construction`, `disused` và `abandoned`. Các thuộc tính này được dùng làm cơ sở để kiểm tra luật.

Bảng 2: Các luật IF-THEN được sử dụng trong Logic Filter

Luật	Điều kiện IF	Hành động THEN
R1	<code>access</code> , <code>vehicle</code> , <code>motor_vehicle</code> , <code>motorcar</code> , <code>hgv</code> hoặc <code>goods</code> thuộc nhóm hạn chế như <code>no</code> , <code>private</code> , <code>restricted</code> , <code>permit</code> , <code>bus</code> , <code>psv</code>	Loại cạnh khỏi đồ thị vì xe giao hàng không được phép đi qua
R2	Cạnh có tag ngoại lệ cho giao hàng như <code>delivery=yes</code> , <code>hgv=delivery</code> , <code>goods=delivery</code>	Giữ cạnh, vì xe giao hàng được phép đi qua dù có một số hạn chế chung
R3	<code>vehicle_weight</code> lớn hơn giới hạn trong <code>maxweight</code> hoặc <code>maxgcweight</code>	Loại cạnh vì xe vượt quá tải trọng cho phép
R4	<code>highway</code> thuộc nhóm <code>construction</code> , <code>proposed</code> , <code>abandoned</code> , <code>planned</code> , <code>razed</code> , <code>demolished</code> , <code>disused</code>	Loại cạnh vì đường đang thi công, chưa hoạt động hoặc không còn sử dụng
R5	Thuộc tính <code>construction</code> có giá trị khác <code>no</code> , <code>none</code> , <code>false</code> , <code>0</code>	Loại cạnh vì đoạn đường đang trong trạng thái thi công
R6	<code>length</code> bị thiếu, không phải số hoặc nhỏ hơn hay bằng 0	Loại cạnh vì dữ liệu hình học không hợp lệ

Hàm chính của module là `filter_graph(graph_data, vehicle_weight)`. Hàm này nhận đầu vào là dữ liệu đồ thị từ module bản đồ và tải trọng xe giao hàng, sau đó trả về một bản sao của đồ thị đã được loại bỏ các cạnh vi phạm luật. Việc dùng bản sao giúp bảo toàn đồ thị gốc, hỗ trợ việc kiểm thử và so sánh giữa dữ liệu trước và sau khi lọc.

Một điểm quan trọng trong thiết kế là module không chỉ đọc thuộc tính trực tiếp từ cạnh trong `MultiDiGraph`, mà còn đồng bộ dữ liệu từ bảng `graph_data["edges"]`. Cơ chế này giúp các luật logic vẫn hoạt động khi một số thuộc tính bị thiếu trong `NetworkX` graph nhưng vẫn tồn tại trong bảng metadata cạnh. Nhờ đó, module có thể tích hợp ổn định hơn với dữ liệu bản đồ do module hệ thống cung cấp.

Sau khi lọc, các thống kê được lưu trong `cleaned_graph.graph["logic_filter_stats"]`, bao gồm số cạnh ban đầu, số cạnh bị loại theo từng nguyên nhân, tổng số cạnh bị loại, số cạnh còn lại, tải trọng xe và số cạnh được đồng bộ metadata. Các thống kê này được dùng trong phần thực nghiệm để đánh giá tác động của Logic Filter lên đồ thị đầu vào.

2.3. Uncertainty Modeling – Mạng Bayes

Mạng Bayes được sử dụng để mô hình hóa yếu tố bất định của giao thông, cụ thể là xác suất kẹt xe trên từng đoạn đường. Thay vì gán chi phí cố định cho mỗi cạnh, hệ thống ước lượng xác suất kẹt xe dựa trên điều kiện thời gian thực như thời tiết, khung giờ và đặc trưng vật lý của đoạn đường.

Trong module `bayes_model.py`, mạng Bayes được thiết kế theo cấu trúc rút gọn như sau:

Bảng 3: Cấu trúc phụ thuộc của mạng Bayes

Biến cha (Parent)		Biến con (Child)
Weather	→	RoadCondition
TimeOfDay	→	TrafficDemand
RoadType	→	TrafficDemand, EffectiveCapacity, AccidentRisk
MaxSpeed	→	EffectiveCapacity
Length	→	AccidentRisk
RoadCondition, TrafficDemand, EffectiveCapacity, AccidentRisk	→	Congestion

Dựa trên cấu trúc ở Bảng 3, các biến trong mạng Bayes được chia thành ba nhóm chính:

- **Biến quan sát (Observed variables):** Weather, TimeOfDay, RoadType, MaxSpeed, Length.
- **Biến ẩn (Hidden variables):** RoadCondition, TrafficDemand, EffectiveCapacity, AccidentRisk.
- **Biến truy vấn (Query variable):** Congestion, biểu diễn xác suất một cạnh bị kẹt xe.

Mục tiêu suy luận của mạng Bayes là tính xác suất:

$$P(\text{Congestion} = \text{True} \mid \text{Weather}, \text{TimeOfDay}, \text{RoadType}, \text{MaxSpeed}, \text{Length})$$

Kết quả suy luận được lưu dưới tên `p_congestion`. Giá trị này sau đó được truyền sang module học máy như một đặc trưng đầu vào, giúp mô hình dự đoán thời gian di chuyển phản ánh được điều kiện giao thông động thay vì chỉ dựa vào độ dài đường và tốc độ giới hạn.

2.4. Mô hình Học máy

Module Học máy được thiết kế để dự đoán thời gian di chuyển trên từng cạnh của đồ thị, ký hiệu là `travel_time_min`. Giá trị này đóng vai trò là trọng số động cho thuật toán tìm kiếm lộ trình, thay thế cho cách tiếp cận dùng khoảng cách vật lý tĩnh.

Mô hình nhận đầu vào là các đặc trưng của từng cạnh đường đi, bao gồm cả đặc trưng hình học và đặc trưng ngữ cảnh giao thông:

$$\hat{t} = f(\underbrace{\text{length_m, maxspeed_kmh, road_type}}_{\text{đặc trưng vật lý}}, \underbrace{\text{weather, is_peak_hour}}_{\text{điều kiện môi trường}}, \underbrace{\text{congestion_prob}}_{\text{đầu ra Bayes}})$$

Trong đó:

- `length_m`: độ dài đoạn đường,
- `maxspeed_kmh`: tốc độ tối đa của tuyến đường,
- `road_type`: loại đường,
- `weather`: điều kiện thời tiết hiện tại,
- `is_peak_hour`: trạng thái giờ cao điểm,
- `congestion_prob`: xác suất ùn tắc do module Bayes cung cấp.

Biến mục tiêu của mô hình là:

`travel_time_min`

Trong giai đoạn baseline, nhóm huấn luyện và so sánh hai mô hình:

- **Linear Regression**
- **Decision Tree Regressor**

Sau khi huấn luyện, mô hình có sai số RMSE thấp hơn sẽ được chọn làm mô hình chính. Artifact mô hình được lưu tại thư mục `features/` dưới dạng `.npy`, còn các biểu đồ và bảng đánh giá được lưu trong thư mục `modules/ml/artifacts/`.

Đầu ra của module Học máy là bảng `weighted_edges` với định dạng:

`u, v, key, travel_time_min`

Bảng này được truyền sang module tìm kiếm để cập nhật trọng số cạnh trong đồ thị `MultiDiGraph`, giúp thuật toán A^* tối ưu theo thời gian di chuyển dự đoán thay vì chỉ dựa vào khoảng cách.

3. Các Thuật toán Sử dụng

3.1. Thuật toán Tìm đường – A^*

Thuật toán A^* được sử dụng để tìm đường đi ngắn nhất (theo thời gian) giữa hai node trên đồ thị:

- **Hàm heuristic (Haversine):**

$$h(n, \text{goal}) = \frac{d_{\text{Haversine}}(n, \text{goal})}{v_{\text{max}}} \quad (\text{phút})$$

Hàm này *admissible* vì $h(n) \leq h^*(n)$ – thời gian đường chim bay luôn nhỏ hơn thực tế.

- **Hàm chi phí:** `travel_time_min` được dự đoán bởi mô hình Decision Tree.
- **Đồ thị:** `nx.MultiDiGraph` từ `OSMnx`, đã qua Logic Filter.

3.2. Giải quyết TSP – Brute-force Permutation + CSP

Để tìm thứ tự giao hàng tối ưu thỏa mãn Time Windows:

1. Xây dựng ma trận khoảng cách $D[u][v]$ giữa tất cả cặp điểm quan trọng bằng A^* .

2. Duyệt qua tất cả $n! = 4! = 24$ hoán vị của các điểm đích.
3. Với mỗi hoán vị, kiểm tra ràng buộc CSP:

$$\text{arrival_time}(d) = \max(\text{current_time} + D[\text{current}][d], \text{tw_start}[d])$$

Hoán vị bị loại nếu $\text{arrival_time}(d) > \text{tw_end}[d]$ với bất kỳ điểm d nào.

4. Chọn hoán vị hợp lệ có tổng thời gian nhỏ nhất.

3.3. Mạng Bayes – Conditional Probability Tables

Module Bayes sử dụng các bảng xác suất có điều kiện (Conditional Probability Tables – CPT) để lượng hóa ảnh hưởng của thời tiết, khung giờ, loại đường, tốc độ tối đa và chiều dài đường lên xác suất kẹt xe.

3.3.1. CPT cho RoadCondition

Điều kiện mặt đường phụ thuộc trực tiếp vào thời tiết:

Bảng 4: CPT cho RoadCondition

Weather	P(RoadCondition = bad)
clear	0.08
rain	0.45

Khi trời mưa, xác suất mặt đường xấu tăng mạnh, từ đó làm tăng khả năng xảy ra ùn tắc trên các cạnh của đồ thị.

3.3.2. CPT cho TrafficDemand

Nhu cầu giao thông phụ thuộc vào khung giờ và loại đường. Các tuyến đường trực chính thường có nhu cầu giao thông cao hơn, đặc biệt trong giờ cao điểm.

Bảng 5: CPT cho TrafficDemand

TimeOfDay	RoadType	P(TrafficDemand = high)
normal	expressway	0.24
normal	arterial	0.32
normal	collector	0.28
normal	local	0.20
peak	expressway	0.55
peak	arterial	0.70
peak	collector	0.62
peak	local	0.48

3.3.3. EffectiveCapacity và AccidentRisk

Khả năng thông hành hiệu dụng của đường được tính từ xác suất nền theo loại đường, sau đó điều chỉnh theo tốc độ tối đa và thời tiết. Ví dụ, đường có tốc độ thấp hoặc trời mưa sẽ làm tăng xác suất **EffectiveCapacity = low**. Ngược lại, đường có tốc độ cao như expressway sẽ có xác suất năng lực thông hành thấp nhỏ hơn.

Rủi ro sự cố được tính dựa trên loại đường, thời tiết và chiều dài đoạn đường. Cạnh càng dài thì xác suất xảy ra sự cố càng cao, đặc biệt trong điều kiện mưa.

3.3.4. CPT cho Congestion

Node Congestion phụ thuộc vào bốn biến ẩn: **TrafficDemand**, **EffectiveCapacity**, **RoadCondition** và **AccidentRisk**. Bảng CPT gồm $2^4 = 16$ tổ hợp trạng thái.

Bảng 6: Một số trường hợp tiêu biểu trong CPT của Congestion

HighDemand	LowCapacity	BadCondition	HighAccident	P(Congestion = True)
False	False	False	False	0.04
True	False	False	False	0.18
False	True	False	False	0.16
True	True	False	False	0.42
True	False	True	True	0.50
True	True	False	True	0.63
False	True	True	True	0.47
True	True	True	True	0.82

Trường hợp tất cả yếu tố bất lợi cùng xảy ra, tức nhu cầu giao thông cao, năng lực thông hành thấp, mặt đường xấu và rủi ro sự cố cao, xác suất kẹt xe đạt mức lớn nhất. Điều này phản ánh đúng trực giác giao thông trong môi trường đô thị.

3.3.5. Công thức suy luận

Vì các biến **TrafficDemand**, **EffectiveCapacity**, **RoadCondition** và **AccidentRisk** không được quan sát trực tiếp, module thực hiện marginalization trên toàn bộ tổ hợp trạng thái của các biến ẩn:

$$P(C = True \mid E) = \sum_{td, ec, rc, ar} P(C = True \mid td, ec, rc, ar) P(td \mid TimeOfDay, RoadType) P(ec \mid RoadType)$$

Trong đó:

$$E = \{Weather, TimeOfDay, RoadType, MaxSpeed, Length\}$$

Hàm `_infer_congestion_probability()` hiện thực công thức trên bằng cách duyệt qua 16 tổ hợp của bốn biến ẩn, tính xác suất từng trạng thái ẩn và cộng dồn xác suất kẹt xe tương ứng.

Đầu ra của module có thể ở hai dạng. Thứ nhất, `compute_graph_congestion()` trả về bảng xác suất kẹt xe theo từng cạnh với các cột `u`, `v`, `key`, `p_congestion`, `weather`, `time_of_day` và `observed_at`. Thứ hai, `compute_congestion_matrix()` trả về ma trận xác suất kẹt xe giữa các node, phục vụ trực tiếp cho pipeline tích hợp và re-planning.

4. Thực nghiệm

4.1. Thiết lập Thực nghiệm

Bảng 7: Tham số thực nghiệm

Tham số	Giá trị
Khu vực bản đồ	ĐH Bách Khoa – Lý Thường Kiệt, Q.10, TP.HCM
Bán kính tải bản đồ	2.000m
Số nodes / edges	2.988 nodes / 7.220 edges
Số điểm giao hàng	4 điểm
Tải trọng xe	2,5 tấn
Thời gian xuất phát	480 phút (8:00 sáng)
Điều kiện ban đầu	rain + peak + tắc đường cục bộ
Điều kiện re-planning	clear + normal

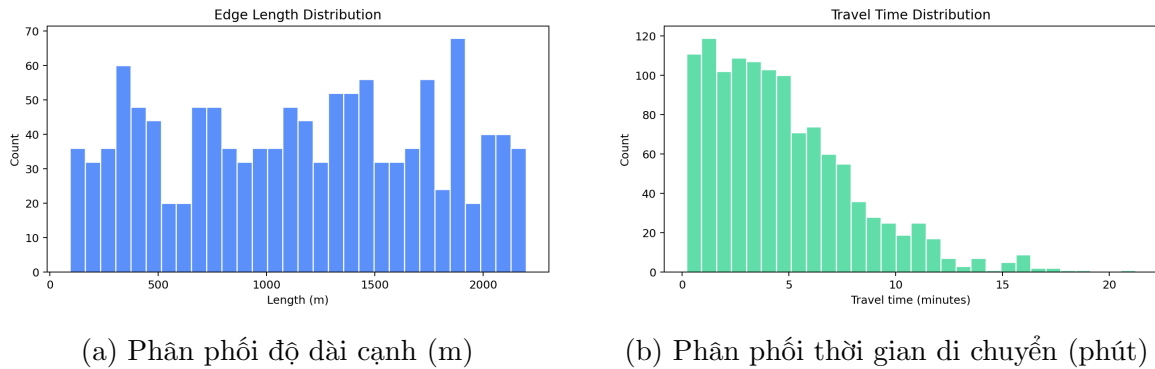
4.2. Phân tích Dữ liệu (EDA)

Dataset huấn luyện ML gồm **1.200 mẫu** với 7 đặc trưng, được tạo từ dữ liệu cạnh OSMnx thực tế kết hợp mô phỏng nhân.

Bảng 8: Thống kê mô tả dataset huấn luyện

Đặc trưng	Min	25%	Median	75%	Max
length_m (m)	91,5	653,9	1.168,6	1.665,2	2.198,1
maxspeed_kmh (km/h)	20,0	31,8	39,0	49,0	75,0
congestion_prob	0,060	0,260	0,408	0,559	0,760
travel_time_min (phút)	0,20	2,07	4,03	6,56	21,23

Phân phối loại đường trong dataset: **residential** (29,7%), **tertiary** (24,0%), **primary** (17,0%), **secondary** (15,7%), **service** (8,7%), **motorway** (5,0%) – phản ánh cấu trúc đường bộ thực tế trong khu vực đô thị.



Hình 1: Phân tích phân phối dữ liệu huấn luyện

Nhận xét: Phân phối độ dài cạnh tương đối đều trong khoảng 100–2.000m, phản ánh sự đa dạng của mạng lưới đường đô thị. Phân phối thời gian di chuyển lệch phải (right-skewed) với phần lớn cạnh có thời gian di chuyển dưới 5 phút, phù hợp với bán kính bản đồ 2km.

4.3. Kết quả Thực nghiệm

4.3.1. Kết quả Logic Filter

Bảng 9: Kết quả lọc đồ thị giao thông

Chỉ số	Giá trị
Số cạnh ban đầu	7.220
Số cạnh bị loại	16
Số cạnh còn lại	7.204
Tỷ lệ lọc	0,22%

16 cạnh bị loại chủ yếu do vi phạm tải trọng hoặc thuộc loại đường đang thi công trong khu vực bán kính 2km.

4.3.2. Kết quả Mạng Bayes Realtime

Sau khi đồ thị được lọc bởi Logic Filter, module Bayes được sử dụng để tính xác suất kẹt xe cho từng cạnh trong điều kiện môi trường hiện tại. Trong kịch bản thực nghiệm ban đầu, hệ thống được thiết lập với điều kiện **rain + peak**, tương ứng với trường hợp trời mưa và giờ cao điểm. Đây là điều kiện bất lợi, làm tăng đồng thời xác suất mặt đường xấu, nhu cầu giao thông cao và rủi ro sự cố.

Đầu ra của module Bayes gồm hai thành phần chính:

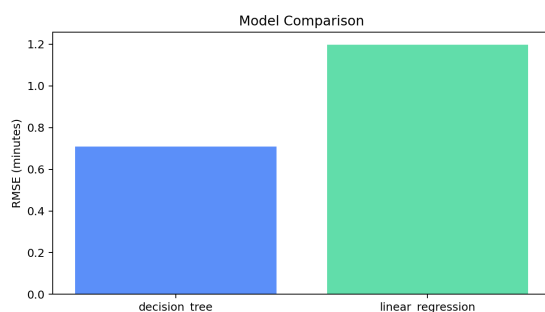
- **Bảng xác suất theo cạnh:** chứa các cột `u`, `v`, `key`, `p_congestion`, `weather`, `time_of_day`, `observed_at`. Bảng này được dùng làm feature frame cho module ML.
- **Ma trận xác suất kẹt xe:** biểu diễn xác suất kẹt xe giữa các node trong đồ thị. Với các cạnh song song giữa cùng một cặp node, hệ thống dùng chiến lược `max` để lấy xác suất cao nhất, nhằm bảo toàn trường hợp rủi ro nhất khi đánh giá kết nối giữa hai node.

Bảng 10: Vai trò của đầu ra Bayes trong pipeline thực nghiệm

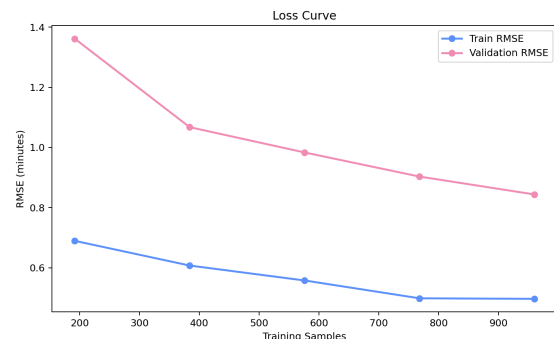
Đầu ra	Cách sử dụng trong hệ thống
<code>p_congestion</code> theo từng cạnh	Được đưa vào dataset đặc trưng của ML để dự đoán <code>travel_time_min</code>
Ma trận xác suất kẹt xe	Được cập nhật lại khi môi trường thay đổi, phục vụ re-planning
Metadata <code>weather</code> , <code>time_of_day</code> , <code>observed_at</code>	Lưu trạng thái môi trường tại thời điểm suy luận, hỗ trợ so sánh trước và sau cập nhật

Kết quả kiểm thử riêng của module Bayes cho thấy xác suất kẹt xe trong điều kiện `rain + peak` cao hơn so với `clear + normal`. Điều này phù hợp với kỳ vọng vì mưa làm tăng khả năng mặt đường xấu và rủi ro sự cố, trong khi giờ cao điểm làm tăng nhu cầu giao thông. Nhờ đó, trọng số cạnh sau khi đi qua ML phản ánh tốt hơn điều kiện giao thông động.

4.3.3. Kết quả So sánh Mô hình ML



(a) So sánh RMSE hai mô hình



(b) Loss curve của Decision Tree

Hình 2: Kết quả huấn luyện mô hình ML

Bảng 11: So sánh hiệu năng mô hình

Mô hình	RMSE (phút)	Kết quả
Decision Tree Regressor	0,71	Được chọn
Linear Regression	1,20	–

Kết quả huấn luyện cho thấy mô hình **Decision Tree Regressor** cho hiệu năng tốt hơn đáng kể so với **Linear Regression**. Cụ thể, mô hình Decision Tree đạt RMSE xấp xỉ **0,71 phút**, thấp hơn nhiều so với mức **1,20 phút** của Linear Regression. Vì vậy, Decision Tree được chọn làm mô hình chính để tích hợp vào pipeline hệ thống.

Ngoài RMSE, mô hình được chọn còn đạt các chỉ số đánh giá tốt trên tập kiểm tra, với $MAE = 0,5139$ và $R^2 = 0,9585$. Điều này cho thấy mô hình dự đoán khá sát với nhãn thời gian di chuyển trong tập dữ liệu mô phỏng, đồng thời có khả năng giải thích phần lớn biến thiên của dữ liệu.

4.3.4. Kết quả A* + CSP (Time Windows)

Bảng 12: Kết quả tìm lộ trình ban đầu bằng A* + CSP

Chỉ số	Giá trị
Thứ tự giao hàng tối ưu	dest_B → dest_C → dest_A → dest_D
Tổng thời gian di chuyển	26,1032 phút
Số hoán vị kiểm tra	24 (4!)

4.3.5. Kết quả Tích hợp & Re-planning

Bảng 13: So sánh kết quả trước và sau re-planning

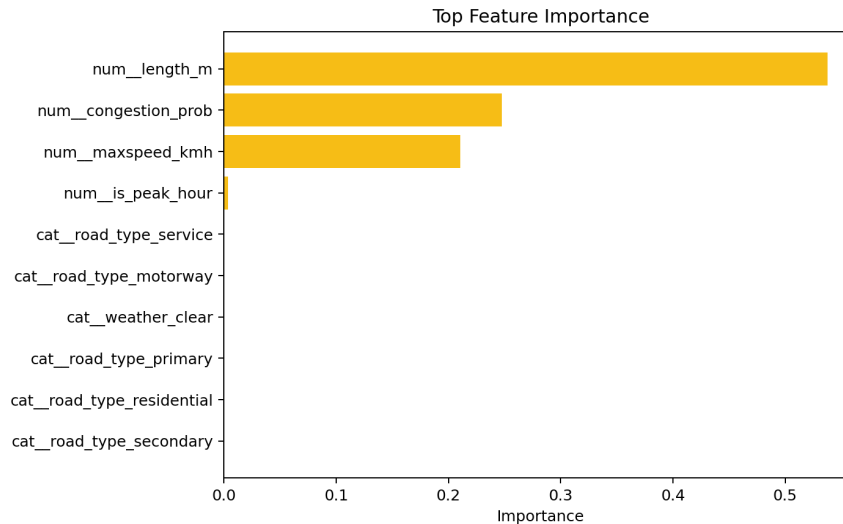
Chỉ số	Kế hoạch ban đầu	Sau re-planning
Điều kiện	rain + peak + tắc cục bộ	clear + normal
Tổng thời gian	22.8258 phút	21,7471 phút
Số node lộ trình	85 nodes	84 nodes

Re-planning được kích hoạt sau khi xe giao thành công điểm dest_B. Với điều kiện môi trường cải thiện (clear + normal) và tắc đường được giải tỏa, hệ thống tính lại lộ trình cho 3 điểm còn lại trong **21,7471 phút** – nhanh hơn so với phần còn lại của kế hoạch cũ.

5. Phân tích Kết quả

5.1. Phân tích Feature Importance

Để giải thích quyết định của mô hình Decision Tree, nhóm sử dụng biểu đồ *feature importance* nhằm đo lường mức độ đóng góp tương đối của từng đặc trưng đầu vào đối với kết quả dự đoán thời gian di chuyển.



Hình 3: Top feature importance của mô hình Decision Tree

Ba đặc trưng quan trọng nhất:

- **length_m ($\approx 0,54$):** Yếu tố vật lý quan trọng nhất – chiều dài cạnh quyết định phần lớn thời gian di chuyển cơ bản.
- **congestion_prob ($\approx 0,25$):** Đầu ra từ mạng Bayes đóng góp đáng kể – xác nhận giá trị của việc tích hợp TV4 vào TV3.
- **maxspeed_kmh ($\approx 0,21$):** Tốc độ giới hạn ảnh hưởng trực tiếp đến thời gian di chuyển tối thiểu.

Đặc biệt, các đặc trưng categorical như **road_type** và **weather** có importance rất thấp sau khi **congestion_prob** đã tổng hợp ảnh hưởng của chúng thông qua mạng Bayes.

5.2. Vai trò của Bayes Logic trong Pipeline

Kết quả feature importance cho thấy **congestion_prob** là một trong ba đặc trưng quan trọng nhất của mô hình Decision Tree, chỉ sau **length_m**. Điều này chứng minh rằng đầu ra của mạng Bayes không chỉ đóng vai trò mô phỏng phụ trợ, mà thực sự ảnh hưởng đến việc dự đoán thời gian di chuyển của toàn hệ thống.

Về mặt pipeline, module Logic và Bayes đóng hai vai trò khác nhau nhưng bổ sung cho nhau. Logic Filter xử lý các ràng buộc cứng, tức những cạnh chắc chắn không được phép đi qua. Ví dụ, nếu đường bị cấm, vượt tải trọng hoặc đang thi công, cạnh đó bị loại khỏi đồ thị trước khi tìm kiếm. Ngược lại, mạng Bayes xử lý các ràng buộc mềm và yếu tố bất định. Một cạnh không bị cấm vẫn có thể có xác suất kẹt xe cao trong điều kiện mưa hoặc giờ cao điểm, và xác suất này được truyền sang ML để làm tăng chi phí di chuyển dự đoán.

Có thể tóm tắt vai trò của hai thành phần như sau:

Bảng 14: So sánh vai trò của Logic Filter và Bayes Model

Thành phần	Loại tri thức xử lý	Tác động lên pipeline
Logic Filter	Ràng buộc chắc chắn, có tính đúng/sai rõ ràng	Loại bỏ cạnh không hợp lệ, giảm không gian tìm kiếm cho A*
Bayes Model	Bất định giao thông, phụ thuộc thời tiết và khung giờ	Tính p_congestion , cập nhật feature cho ML và hỗ trợ re-planning

Nhờ sự kết hợp này, hệ thống không chỉ tìm đường trên đồ thị thực tế mà còn có khả năng thích nghi với điều kiện môi trường thay đổi. Khi trạng thái chuyển từ **rain + peak** sang **clear + normal**, module Bayes cập nhật lại xác suất kẹt xe, ML dự đoán lại thời gian di chuyển, và A* tìm lại lộ trình tối ưu từ vị trí hiện tại.

5.3. Phân tích Loss Curve

Loss curve được sử dụng để đánh giá khả năng tổng quát hóa của mô hình khi kích thước dữ liệu huấn luyện tăng dần. Trong thực nghiệm này, nhóm sử dụng learning curve với chỉ số RMSE trên tập huấn luyện và tập validation để quan sát xu hướng hội tụ của mô hình Decision Tree.

Loss curve (Hình 3b) cho thấy:

- Cả Train RMSE và Validation RMSE đều giảm khi tăng số mẫu huấn luyện – mô hình tiếp tục cải thiện khi có thêm dữ liệu.
- Khoảng cách giữa Train RMSE (0,49) và Validation RMSE (0,85) tương đối ổn định – mô hình không bị overfitting nghiêm trọng.
- Validation RMSE vẫn có xu hướng giảm ở 960 mẫu – cho thấy có thể cải thiện thêm với dataset lớn hơn.

5.4. Hiệu quả Re-planning

Kết quả re-planning cho thấy hệ thống hoạt động đúng như thiết kế: sau khi giao thành công điểm đầu tiên và nhận được tín hiệu môi trường cải thiện, `DeliveryCoordinator` tự động kích hoạt lại pipeline Bayes $\rightarrow$ ML $\rightarrow$ A* để tìm lộ trình tối ưu mới từ vị trí hiện tại. Việc tích hợp `set_local_congestion` / `clear_local_congestion` cho phép mô phỏng tắc đường điểm một cách linh hoạt mà không cần sửa đổi các module con.

6. Kết luận

6.1. Tóm tắt Đóng góp

Nhóm đã xây dựng thành công một hệ thống AI tích hợp đầy đủ 5 thành phần theo yêu cầu học phần:

Bảng 15: Tóm tắt các thành phần AI đã triển khai

	Thành phần	Triển khai
A	Tìm kiếm	A* với heuristic Haversine trên đồ thị OSMnx thực tế
B	Heuristic/CSP	Hàm heuristic admissible + ràng buộc Time Windows
C	Tri thức	Hệ luật IF-THEN trong <code>logic_filter.py</code>
D	Mạng Bayes	<code>BayesCongestionModel</code> với CPT theo loại đường và điều kiện môi trường
E	Học máy	Decision Tree Regressor dự đoán <code>travel_time_min</code>

Đối với thành phần Học máy, nhóm đã xây dựng thành công một module dự đoán thời gian di chuyển theo từng cạnh, có tích hợp đầu ra xác suất ùn tắc từ mạng Bayes. Kết quả thực nghiệm cho thấy mô hình Decision Tree đạt sai số thấp và có thể cung cấp trọng số động hợp lý cho hệ thống tìm đường.

6.2. Hạn chế và Hướng phát triển

Hạn chế:

- Thuật toán TSP dùng brute-force $O(n!)$ – không khả thi với $n > 10$ điểm.
- Dataset huấn luyện ML được tạo từ mô phỏng, chưa dùng dữ liệu giao thông lịch sử thực tế.
- Mạng Bayes dùng CPT tĩnh, chưa cập nhật online từ dữ liệu thực tế.

Hướng phát triển:

- Thay brute-force TSP bằng thuật toán tiến hóa (GA) hoặc reinforcement learning.
- Tích hợp dữ liệu giao thông thời gian thực (Google Maps API, HERE API).
- Mở rộng sang multi-vehicle routing với nhiều xe và kho hàng.

7. Phân công Công việc

Bảng 16: Phân công công việc nhóm

Thành viên	Tỷ lệ	Công việc đảm nhận
Vạn Trường Thành (2353107)	25%	Module Integration (<code>coordinator.py</code>), notebook <code>main_pipeline.ipynb</code> , tài liệu kỹ thuật
Lê Minh Trí (2353229)	25%	Module Search (<code>astar.py</code>), CSP Time Windows, mô hình hóa bài toán
Nguyễn Minh Khôi (2352618)	25%	Module ML (<code>train.py</code> , <code>predictor.py</code>), EDA, phân tích so sánh mô hình
Trần Nguyễn Nhất Thông (2353149)	25%	Module Bayes Logic (<code>bayes_model.py</code> , <code>logic_filter.py</code>), biểu diễn tri thức

8. Mã nguồn & Tài nguyên

Toàn bộ mã nguồn, notebook và báo cáo của dự án được công bố công khai tại:

- **GitHub Repository:** <https://github.com/thanh081205/Delivery-Routing-Optimization>
- **Google Colab Notebook:** https://colab.research.google.com/github/thanh081205/Delivery-Routing-Optimization/blob/main/notebooks/main_pipeline.ipynb